

Deep Reinforcement Learning for Bomberman

Theory, Training Methodology, and Evaluation of the RUEHL_BASED_AGENT

[Name]

[Course / institution]

September 16, 2026

Abstract

Bomberman combines navigation, delayed rewards, and decisions that can place the agent itself in danger. This report describes the RUEHL_BASED_AGENT, which estimates action values using a convolutional neural network with a dueling output architecture. It introduces the Double Deep Q-Network (Double DQN) learning rule and explains how board features, scalar features, and action masking support decision making. The training methodology is organized around a curriculum from coin collection to crate destruction and finally play against opponents. An evaluation framework distinguishes navigation, bomb use, and competitive performance. This draft documents the available implementation; training hyperparameters and numerical results remain to be completed from the experiment records.

Draft status. The implementation descriptions below are based on `Networks.py`, `callbacks.py`, `train.py`, and the project’s environment configuration. The training code now confirms the Double DQN update, reward profiles, and model-selection procedure. Numerical hyperparameters from `Hyperparams.prm` and empirical results remain to be supplied; the available log directory is empty.

1 Theoretical Background and Agent Architecture

1.1 Bomberman as a reinforcement learning problem

At each time step, the agent observes the game, chooses an action, and receives feedback through game events. Its action set is

$$\mathcal{A} = \{\text{UP}, \text{RIGHT}, \text{DOWN}, \text{LEFT}, \text{WAIT}, \text{BOMB}\}.$$

An episode corresponds to a round of play from the agent’s perspective and ends when it dies or the round terminates. The configured board has 17×17 tiles, with a maximum of four agents and 400 steps per round. The aim of the game is to collect coins and eliminate opponents. Coins yield points immediately, whereas placing a bomb can produce a benefit only after its timer expires. The agent must therefore consider future consequences rather than only the next reward.

The usual theoretical model is a Markov decision chain with states, actions, transition probabilities, rewards, and discount factor γ . The state of the game s_t includes the positions of all agents, bomb timers, and revealed coins, as well as any other relevant game information needed to make decisions. To make optimal decisions, the next action should be chosen based on the expected future rewards, considering both immediate and delayed consequences, captured by the action-value function defined as

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{k=0}^{T-t-1} \gamma^k r_{t+k+1} \mid s_t = s, a_t = a \right]. \quad (1)$$

Here, r_{t+1} is the learning reward, T is the episode’s terminal time, and π is the policy. A larger discount factor gives delayed outcomes greater weight. For example, a useful bomb placement may deserve a high value even if the agent must first move away and wait for the explosion.

1.2 From Q-learning to deep Q-learning

A deep Q-network (DQN) uses a neural network to approximate the action-value function, allowing it to generalize across similar states and handle large state spaces. For a given input state s the network outputs a vector of action values $Q(s, a, \theta)$ for all possible actions $a \in \mathcal{A}$.

The learning process updates the network parameters θ using stochastic gradient descent

$$\theta_{t+1} = \theta_t + \alpha \left(Y_t^Q - Q(S_t, A_t, \theta_t) \right) \nabla_{\theta} Q(S_t, A_t, \theta_t), \quad (2)$$

where α is the learning rate and Y_t^Q is the target value for the Q-network at time step t defined as

$$Y_t^Q = r_{t+1} + \gamma \max_a Q(S_{t+1}, a, \theta_t). \quad (3)$$

Previous steps get stored in the *experience replay* buffer, which allows the network to sample minibatches of past transitions for training, reducing the correlation between consecutive updates. Furthermore the target network with parameters θ^- is used to compute the target values, and it is updated every τ steps from the online network’s parameters such that $\theta^- = \theta_t$. The target used in the update then becomes

$$Y_t^Q = r_{t+1} + \gamma \max_a Q(S_{t+1}, a, \theta^-), \quad (4)$$

where θ^- are the parameters of the target network.

1.3 Double DQN: separating selection and evaluation

The maximum in Equation (4) selects and evaluates an action using the same estimates. The issue with this method is that is prone to overestimation of action values, leading to bias in the learned Q-values. Van Hasselt et al. show that this overestimation also occurs with deep networks and propose Double DQN to reduce it [3].

They therefore propose the Double DQN approach, which uses the online network with parameters θ to select the next action and the target network with parameters θ' to evaluate that action. These two networks are then updated independently, by assigning the change to either of the networks randomly during training. Important is that the changing the roles of the online and target networks is done in a symmetric manner. The double DQN target is defined as

$$Y_t^{\text{DoubleDQN}} = r_{t+1} + \gamma Q\left(s_{t+1}, \arg \max_a Q(s_{t+1}, a, \theta_t); \theta'\right), \quad (5)$$

and note that action selection in the $\arg \max$ still uses the online parameter θ_t . The best action of the greedy policy is still estimated using the current values, but the policy networks θ' is used to provide an unbiased evaluation of that action’s value.

In Bomberman, a relevant motivation is to avoid overvaluing risky actions whose apparent payoff is based on inaccurate predictions. The double DQN approach is beneficial in this context, as it helps the agent get more accurate value estimates for risky actions, leading to safer and more effective decision-making.

1.4 Input representation

The function `state_to_features` converts the observation into seven spatial maps and ten scalar values. The board input has shape $7 \times 17 \times 17$ and uses game information directly rather than rendered screen pixels. Table 1 summarizes its channels.

From these standard features, one can construct further informative representations, such as combining bomb locations with the explosion map to a combined **danger map**.

The scalar input consists of the number of rounds, and steps played so far in the game. Additionally it is useful for training to provide information about the agent’s bomb availability,

Channel	Encoding
Walls	Binary map of indestructible walls.
Crates	Binary map of destructible crates.
Coins	Binary map of currently visible coins.
Bombs	Binary map of placed bomb locations.
Explosion map	Integer map of active explosions; the value represents the remaining duration.
Own position	Onehot encoding of the agent’s tile
Opponents	Onehot encoding of each opponent’s tile.

Table 1: Spatial features implemented in `callbacks.py`.

Channel	Encoding
Bomb availability	Binary value indicating if the agent can place a bomb.
Number of coins normalised	Float32 value representing the number of all collectable coins divided by the total number of coins.
Number of opponents	Integer value representing the number of opponents.
Current step	Integer value representing the current step in the game.
Previous action	One-hot encoding of the previous action taken by the agent.
Own position	Onehot encoding of the agent’s tile
Opponents	Onehot encoding of each opponent’s tile.

Table 2: Scalar features implemented in `callbacks.py`.

the number of remaining coins, the number of remaining opponents, the current step, and the previous action. Therefore we defined

By providing the agent with absolute information about the game state, such as the number of remaining coins, opponents, and the current step, the network can make more informed decisions.

1.5 Neural network architecture

Given the combination of spatial and scalar features, it is reasonable to use a network architecture that processes these two types of inputs separately before combining them for decision making. The spatial branch contains four convolutional layers with ReLU activations. Dilated convolutions expand the area visible to a filter without increasing its number of weights, while the third convolution reduces the spatial resolution. Table 3 gives the dimensions derived from `Networks.py`; the batch dimension is omitted.

Component	Operation	Output shape
Board input	Seven spatial channels	$7 \times 17 \times 17$
Convolution 1	3×3 , 32 filters, stride 1, padding 1	$32 \times 17 \times 17$
Convolution 2	3×3 , 32 filters, dilation 2, padding 2	$32 \times 17 \times 17$
Convolution 3	3×3 , 32 filters, stride 2, padding 1	$32 \times 9 \times 9$
Convolution 4	3×3 , 32 filters, dilation 2, padding 2	$32 \times 9 \times 9$
Flatten	Flatten the board features	2592
Scalar branch	Linear $10 \rightarrow 32$, ReLU	32
Concatenation	Join spatial and scalar features	2624
Shared layers	Linear $2624 \rightarrow 128 \rightarrow 128$, ReLU after each	128
Value head	Linear $128 \rightarrow 1$	1
Advantage head	Linear $128 \rightarrow 6$	6

Table 3: Architecture of the RUEHL network for the configured board size.

Each output of the last convolution has a theoretical receptive field of 17×17 input tiles, including padding near the boundary. The flattened features subsequently connect to shared fully

connected layers.

Complementing the double DQN strategy, one can employ a dueling network architecture to separately estimate state values and action advantages, which can lead to more stable learning and improved policy evaluation. In the dueling DQN architecture, the action-value function is not represented directly by a single network output. Instead, the network decomposes the estimate of $Q(s, a)$ into two components: a state-value function $V_\theta(s)$ and an action-dependent advantage function $A_\theta(s, a)$ [4]. The state-value function represents the overall quality of being in state s , independently of which action is selected, whereas the advantage function describes the relative benefit of choosing a particular action a in that state.

A naive decomposition of the form

$$Q_\theta(s, a) = V_\theta(s) + A_\theta(s, a)$$

is not identifiable. In particular, an arbitrary constant c can be added to the value function and subtracted from all advantage values without changing Q_θ :

$$V_\theta(s) + c + (A_\theta(s, a) - c) = V_\theta(s) + A_\theta(s, a).$$

Consequently, the network would have no unique way of determining which part of the predicted action value should be represented by V_θ and which part should be represented by A_θ .

To remove this ambiguity, the dueling architecture combines the two streams by subtracting the mean advantage over all available actions,

$$Q_\theta(s, a) = V_\theta(s) + A_\theta(s, a) - \frac{1}{|\mathcal{A}|} \sum_{b \in \mathcal{A}} A_\theta(s, b). \quad (6)$$

This centers the effective advantage values around zero. An important consequence of this parameterization is obtained by averaging $Q_\theta(s, a)$ over all actions:

$$\frac{1}{|\mathcal{A}|} \sum_{a \in \mathcal{A}} Q_\theta(s, a) = V_\theta(s).$$

Thus, under this aggregation rule, the value stream represents the mean predicted action value in a state, while the centered advantage stream determines how the individual action values differ from this common baseline. In particular, because both $V_\theta(s)$ and $\bar{A}_\theta(s)$ are independent of the selected action,

$$\arg \max_a Q_\theta(s, a) = \arg \max_a A_\theta(s, a).$$

Hence, the advantage stream determines the relative ranking of actions, whereas the value stream controls the overall level of their Q-values.

The motivation for this decomposition is that in many states the precise choice of action has only a small effect on the expected return. A conventional DQN must nevertheless estimate a separate $Q(s, a)$ for every action. The dueling architecture allows the network to learn features that characterize whether a state is generally favorable or unfavorable through $V_\theta(s)$, while using the advantage stream only to represent differences between actions. This can make value estimation more efficient in states where several actions have similar consequences [4].

The two quantities are not trained using separate value and advantage targets. Instead, they are combined according to Eq. (6) to obtain $Q_\theta(s, a)$, and the usual temporal-difference loss is applied to this resulting Q-value. Gradients then propagate through the aggregation operation into both the value and advantage streams. The decomposition is therefore an architectural parameterization of the Q-function rather than the introduction of two independently supervised learning objectives.

2 Training Methodology

2.1 Exploration with an epsilon-greedy policy

The agent implements an ϵ -greedy policy. Let $\mathcal{A}_m(s)$ be the actions retained by its mask. During training, the action is sampled as

$$a_t = \begin{cases} \text{a uniform random action from } \mathcal{A}_m(s_t), & \text{with probability } \epsilon_t, \\ \arg \max_{a \in \mathcal{A}_m(s_t)} Q_\theta(s_t, a), & \text{with probability } 1 - \epsilon_t. \end{cases} \quad (7)$$

Exploration supplies experience with actions that the current network may undervalue; exploitation uses what has already been learned. Both branches respect the mask. The random branch can also select the greedy action, so ϵ_t is the probability of random selection, not necessarily the probability of taking a non-greedy action.

In `act`, ϵ_t comes from `epsilon_current`; it defaults to 1.0 in training if that attribute has not been set. Evaluation rounds and normal non-training play use $\epsilon = 0$. The inference code contains no decay schedule. A possible linear schedule, included here only to illustrate the mechanism, is

$$\epsilon_t = \epsilon_{\min} + (\epsilon_{\text{start}} - \epsilon_{\min}) \max \left(0, 1 - \frac{t}{T_{\text{decay}}} \right). \quad (8)$$

High initial exploration can help discover useful actions, whereas lower exploration later allows more consistent use of the learned policy. Introducing crates or opponents may require renewed exploration of actions that were unhelpful in earlier stages. Useful values for $\epsilon_{\min}$ are typically around 0.05.

2.2 Curriculum: from coins to crates to opponents

The training progression considered in this project starts with navigation, then introduces bomb use, and finally adds opponents. This curriculum is motivated by the difficulty of discovering all three behaviors at once: an inexperienced agent can otherwise die before collecting useful examples of delayed rewards. Table 4 links the progression to the available scenarios.

Scenario	Crate density	Coins	Role in the curriculum
<code>coin-heaven</code>	0	50	Learn movement and collection; the initial stage is intended to run without opponents.
<code>loot-crate</code>	0.75	50	Learn to open routes, reveal coins, and escape after placing bombs.
<code>classic</code>	0.75	9	Combine collection and bomb use with competition against opponents.

Table 4: Configured scenarios and their intended training roles. Crate density is the placement probability before walls and starting areas are cleared. Opponents are configured separately from the scenario.

Stage 1: coin collection. In the `coin-heaven` scenario, visible coins give a relatively direct objective: move through the maze and collect them efficiently. The agent can learn spatial relationships between its position, walls, and rewards. For this initial stage of training, we started the epsilon greedy strategy with $\epsilon = 1$, ensuring maximum exploration. Via reward shaping, the agent receives immediate feedback for collecting coins, and negative feedback for undesirable actions, guiding it towards efficient coin collection.

Stage 2: crate destruction. The `loot-crate` scenario adds destructible obstacles and places coins preferentially underneath them. Successful collection can now require a sequence of actions: approach a crate, place a bomb, reach safety, and return to collect a revealed coin. This stage introduces delayed credit assignment and the conflict between making progress and avoiding self-destruction. Initializing from the pretrained Network on `coin-heaven`, we continued training with substantial epsilon reheating $\epsilon = 0.85$ to encourage exploration in the presence of destructible crates. For this stage, a different reward profile is used, emphasizing successful crate destruction and safe bomb placement, while still rewarding coin collection and very importantly, penalises suicidal actions.

Stage 3: opponents and competitive play. The main goal for this stage is to adapt to less placed coins, while dealing with dynamic opponents. A progression from simpler opponents to stronger ones would let the agent adapt these skills gradually. For this purpose, we iteratively improved the model, by transferring the pretrained network from `loot-crate` to `classic` with slight epsilon reheating $\epsilon = 0.20$, then gradually increased the difficulty of opponents.

2.3 Action masking and hand-designed guidance

The mask is an important part of the effective policy. It first rejects movement into walls, crates, bombs, or opponents. It then filters movement and waiting on active explosions or blast tiles of bombs whose timers are at most one. Bomb placement is allowed only when a bomb is available, its blast could hit a crate or opponent, and a static breadth-first search finds a tile outside its blast within the configured bomb timer.

If no action survives the danger filter, the code falls back to the earlier legal-action mask. If that is also empty, it enables `WAIT`. The danger filter directly checks the movement and wait entries; the bomb entry uses its separate usefulness and escape checks. This is therefore a heuristic filter, not a guarantee of survival. Its static escape search does not predict moving opponents or jointly simulate other bombs over time.

This guidance reduces the exploration burden, but it also contributes to the observed behavior independently of learned Q-values. Results should therefore describe the complete network-plus-mask policy. The Double DQN update in `train.py` also applies the next-state action mask before the online network selects its maximizing action. The target network then evaluates that selected action.

2.4 Reward design: objectives and behavioral guidance

The reward design addresses two difficulties: useful outcomes can be delayed, and actions that create opportunities can also endanger the agent. Rewarding only collected coins and confirmed kills would give little immediate feedback about the actions that made those outcomes possible. The profiles therefore combine rewards for outcomes and intermediate progress with smaller penalties that guide everyday movement. The following rationale explains the intended role of these choices; their effectiveness and numerical weights still require empirical evaluation.

Main rewards define progress and its costs. The main contribution, stored under `major` in the implementation, assigns values to selected game events. Collecting a coin provides a common unit of reward across all three profiles. Killing an opponent receives a larger reward, while death carries a substantial penalty. A separate suicide penalty further emphasizes self-preservation in competitive play. This expresses the trade-off between gaining points and retaining the opportunity to act. Penalizing only death would encourage excessive caution; rewarding only attacks would provide insufficient feedback about their danger. Their combination is intended to encourage productive risks with an escape plan.

The main contribution also includes intermediate incentives: revealing a coin, destroying a crate, or placing a bomb that threatens a useful target. These events shorten the delay between an action and positive feedback. For example, the crate profile rewards opening a route before a hidden coin is collected. Importantly, *main* does not mean identical to the game’s score: these intermediate bonuses are also reward shaping in the broader reinforcement-learning sense. The code’s distinction is operational: main event rewards are accumulated without clipping, whereas the separate **shaping** contribution is bounded. Preserving the main event sum lets an action that destroys several crates receive credit for each one.

Shaping penalties express soft behavioral preferences. The separate shaping terms discourage waiting, entering an active explosion, and, in the competitive profile, immediately reversing movement direction. Their purpose is to discourage inactivity and repeated back-and-forth motion without making these actions impossible. Waiting for an explosion to clear or reversing direction to escape can be necessary, so these penalties should remain small relative to important outcomes. In particular, the danger penalty supplies local feedback about an active explosion; it does not by itself teach prediction of every future blast. Action masking provides additional guidance, as described above.

Clipping controls the influence of frequent feedback. Several shaping events can occur in one transition. Clipping their combined contribution limits their combined penalty on that step, while leaving the main event rewards intact. The limits of 0.05 for **coin** and **loot_exploration**, and 0.15 for **killer**, are small compared with the 1.0 collection reward. This establishes a per-step scale, not a guarantee that shaping cannot dominate a long episode: repeated small penalties still accumulate. The additional step cost favors shorter successful trajectories, but is removed during crate exploration to reduce pressure against lengthy bombing and escape sequences. The completion bonus separately encourages finishing collection rather than leaving the last coins uncollected.

Limits of the design and its justification. These event-based incentives encode useful hypotheses about behavior, but an intermediate reward is only a proxy for success. For instance, repeatedly threatening an opponent can earn placement rewards without producing a kill. Ng et al. show that potential-based shaping can preserve optimal policies under the conditions of their framework [1]; our event bonuses and penalties are not constructed to satisfy that guarantee. Consequently, we assess checkpoints using game outcomes rather than total shaped return, as detailed in the model-selection subsection. Comparisons with individual bonuses or penalties removed would be needed to establish whether each component improves collection, survival, or competitive play.

2.5 Task-specific reward profiles

The environment’s game score is

$$S = N_{\text{coins}} + 5N_{\text{opponents killed}}, \quad (9)$$

whereas the learning reward provides additional feedback for intermediate behaviors. Three task-specific profiles support this progression: **coin**, **loot_exploration**, and **killer**. The environment variable `DQN_REWARD_PROFILE` selects the profile; **standard** is an additional default. All profiles share the same training loop.

Coin collection. The **coin** profile rewards collection and discourages waiting through both a shaping penalty and a small step cost. It gives no direct incentive for crate destruction or bomb placement, focusing the initial task on navigation. Opponent-kill rewards are inherited from the shared reward table but have no effect in a scenario without opponents.

Event or parameter	coin	loot_exploration	killer
Coin collected	1.0	1.0	1.0
Coin revealed	0.1	0.5	0.5
Crate destroyed	0	0.35	0.15
Bomb placed to hit a crate	0	0.05	0.02
Bomb placed to threaten an opponent	0	0	0.75
Opponent killed	6.5	6.5	6.0
Killed by another agent	-1.0	-1.0	-2.0
Self-destruction	-2.0	-2.0	-6.0
Bomb exploded	0	0	0.15
Round survived	0	0	0.75
Waiting (shaping)	-0.08	-0.01	-0.05
Danger (shaping)	-0.02	-0.04	-0.10
Reversing direction (shaping)	0	0	-0.03
Shaping limit c	0.05	0.05	0.15
Step cost λ	0.008	0	0.002

Table 5: Reward profiles in `train.py`. Event values are summed per transition; only the combined shaping contribution is clipped. A zero denotes an event with no assigned reward in that profile.

Crate exploration. The `loot_exploration` profile rewards destroying crates, revealing coins, and placing bombs whose blast reaches a crate. These intermediate rewards provide feedback before the agent eventually collects a coin. Removing the step cost and reducing the waiting penalty allows more time for bomb placement and escape sequences, while the self-destruction penalty retains an incentive to survive.

Competitive play. The `killer` profile emphasizes successful attacks and survival. A bomb that currently threatens an opponent earns an immediate reward of 0.75, while a confirmed kill earns 6.0. Self-destruction costs 6.0, substantially more than death caused by another agent. Smaller crate rewards preserve an incentive to open routes, and surviving the round earns 0.75. The bomb-placement event checks the current blast geometry, not whether the opponent will actually be hit; if both opponents and crates are in range, only the opponent-targeting placement event is added.

For each transition, `reward_from_events` computes

$$r_{t+1} = R_{\text{major}} + \text{clip}(R_{\text{shaping}}, -c, c) - \lambda. \quad (10)$$

Thus, for example, the coin profile’s waiting penalty is limited to -0.05 before subtracting the step cost. The danger event specifically checks whether the new position lies on an active explosion; it is distinct from the prospective danger map used by the policy. On self-destruction, the code applies the dedicated suicide penalty and skips the ordinary death and bomb-explosion contributions, avoiding their accumulation. Independently of the profile, collecting the configured total number of coins while no opponents remain gives a one-time completion bonus of $5 + (\text{remaining steps}/400)$, favoring earlier completion.

2.6 Optimization and training state

The training loop stores transitions in a bounded replay buffer and samples minibatches uniformly without replacement after a configurable warm-up. It implements the Double DQN target from Equation (5): the masked online network selects the next action, and the target network evaluates it. Terminal transitions have no bootstrap contribution. Optimization uses AdamW with a smooth L1 loss, and clips the gradient norm to 5.0. The target network receives a hard copy

of the online weights every `TARGET_UPDATE` gradient updates. Learning occurs at a configured environment-step interval and through additional updates at the end of training rounds.

The replay capacity, batch size, discount factor, learning rate, warm-up, update intervals, and evaluation budget are read from `Hyperparams.prm`. That file is not available in the supplied folder, so its numerical values remain to be documented. These implementation choices matter for reproducibility and stability [2].

Training supports fresh initialization, resuming, and transfer. Resuming with the same reward profile restores the policy, target network, optimizer, and training/evaluation counters from the latest checkpoint, progress from `Hyperparams.prm`, and the replay buffer if its separate file exists. Transfer retains the pretrained policy while starting with a new optimizer, empty replay buffer, and reset progress, including the best selection score. Attempting to resume with a different reward profile automatically uses this transfer behavior, so previously stored rewards do not enter the new profile’s replay buffer. The latest checkpoint and progress file are saved after every round; replay is saved separately every 100 rounds.

2.7 Evaluation and model selection

After `EVAL EVERY TRAINING ROUNDS` training rounds, the agent plays `EVAL ROUNDS` greedy evaluation rounds with $\epsilon = 0$. During these rounds, no transitions are added to replay and no optimization or exploration-decay steps occur. Checkpoints are therefore assessed through the behavior of a fixed policy rather than its exploratory training return.

Let $\bar{S}$ denote mean game score, $\bar{K}$ mean opponent kills per round, and p_{suicide} the fraction of evaluation rounds ending in self-destruction. The selection criterion is

$$J = \begin{cases} \bar{S} + 5\bar{K} - 5p_{\text{suicide}}, & \text{for killer,} \\ \bar{S}, & \text{for the other profiles.} \end{cases} \quad (11)$$

For coin collection and crate exploration, this selects for actual game score rather than accumulated shaping reward. Since game score already includes five points per kill, the `killer` criterion effectively weights mean kills by ten, while explicitly penalizing suicides. Mean coin and crate counts, completion rate, and mean time remaining among completed rounds are also recorded, but do not independently affect selection.

Only a strict improvement in J replaces the policy weights in `z_best-model.pt`; ties retain the previous model. This selected model is preferred during non-training play, whereas the latest checkpoint serves to resume training. Periodic selection is useful because DQN performance can fluctuate even as training continues [2]. The evaluation block supplies a point estimate without an uncertainty-based replacement test, so a separate final test is needed to assess whether an apparent improvement generalizes. Opponent lineups and evaluation budgets should also be held fixed when comparing selection scores.

3 Results and Discussion

3.1 Evaluation protocol

The evaluation should test each curriculum skill as well as final competitive performance. Use fixed checkpoints with learning disabled and $\epsilon = 0$, and record the scenario, opponent lineup, mask configuration, number of rounds, and random seeds. Compare agents on the same evaluation conditions. Final test seeds should be separate from those used to select checkpoints.

Coin collection is best measured by coins collected per round and collection rate. In crate scenarios, also report crates destroyed, useful bomb outcomes, and self-destruction rate. For competitive play, report game score, opponent kills, survival, and a clearly defined win rate. One suitable score-based definition counts a win only when the agent has the unique highest final

score and reports ties separately; being the last survivor is a different measure and should be labeled accordingly.

Useful comparisons include the repository’s `random_agent`, `coin_collector_agent`, and `rule_based_agent`, matched to the relevant scenario. The same final architecture trained from scratch in `classic` would provide a control for the curriculum. A comparison with reduced masking would help distinguish learned behavior from the hand-designed guidance. These are proposed evaluations, not completed runs.

For a metric x , report the number of rounds N , sample mean, and sample standard deviation:

$$\bar{x} = \frac{1}{N} \sum_{i=1}^N x_i, \quad s_x = \sqrt{\frac{1}{N-1} \sum_{i=1}^N (x_i - \bar{x})^2}. \quad (12)$$

Several independent training seeds would additionally reveal variability between learned policies, which repeated games with one checkpoint do not measure. Training-return curves and greedy-evaluation curves should be reported separately because exploration changes the behavior being measured.

3.2 Summary of the available evidence

The supplied source establishes the network architecture, observation encoding, masked action selection, and checkpoint-loading paths. A saved model file is present, but the available files contain no numerical evaluation record. Consequently, this draft cannot yet conclude that the agent improved across stages, outperformed a baseline, or benefited from Double DQN. Table 6 provides a structure for the measurements.

Evaluation setting	Rounds N	Coins mean $\pm$ SD	Game score mean $\pm$ SD	Self-destruction rate
Coin-heaven checkpoint	—	—	—	—
Loot-crate checkpoint	—	—	—	—
Final agent, classic	—	—	—	—
Rule-based baseline, classic	—	—	—	—

Table 6: Results to be filled from evaluation logs. Dashes denote unavailable measurements, not zero. Specify the opponent lineup for every setting and add kills, win rate, and ties for the competitive comparison.

TODO: Fill Table 6, add learning curves with stage boundaries, and summarize representative successful and failed rounds. Identify the checkpoint and evaluation conditions behind every reported number.

3.3 Interpretation and limitations

The main questions are whether navigation transfers to crate environments, whether bomb use yields rewards without excessive self-destruction, and whether those skills remain effective against opponents. Higher collection in the first stage would support a navigation claim; improved crate destruction with lower self-destruction would support more effective bomb use. Competitive success additionally requires evidence against a specified opponent lineup. These outcomes are hypotheses until measured.

Several design choices should inform the interpretation. First, the action mask prevents some mistakes before the network chooses an action, so good survival alone does not demonstrate that safety was learned. Second, danger is represented by a compact intensity map and a static escape heuristic, which may be insufficient for interacting threats. Third, changing the coin

normalization or reward function between stages can change the meaning of inputs or predicted returns. Finally, a curriculum can cause the agent to forget earlier skills, making evaluation on previous scenarios useful.

3.4 Conclusion

The RUEHL_BASED_AGENT combines a convolutional representation of the board with scalar context, dueling value and advantage heads, and a rule-based action mask. Double DQN provides a suitable theoretical learning rule by separating action selection from target evaluation, while a progression from coins to crates and opponents organizes the task into increasingly demanding skills. The available implementation supports a detailed account of how the policy acts. Completing the training configuration and empirical evaluation will allow the final report to establish which skills were learned and how well the resulting agent performs.

References

- [1] Andrew Y. Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the Sixteenth International Conference on Machine Learning*, pages 278–287. Morgan Kaufmann, 1999. URL <https://ai.stanford.edu/~ang/papers/shaping-icml99.pdf>.
- [2] Melrose Roderick, James MacGlashan, and Stefanie Tellex. Implementing the deep Q-network. *arXiv preprint arXiv:1711.07478*, 2017. URL <https://arxiv.org/abs/1711.07478>. Version 1. Local source: 1711.07478v1.pdf.
- [3] Hado van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double Q-learning. *arXiv preprint arXiv:1509.06461*, 2015. URL <https://arxiv.org/abs/1509.06461>. Version 3. Local source: 1509.06461v3.pdf.
- [4] Ziyu Wang, Tom Schaul, Matteo Hessel, Hado van Hasselt, Marc Lanctot, and Nando de Freitas. Dueling network architectures for deep reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pages 1995–2003. PMLR, 2016. URL <https://proceedings.mlr.press/v48/wangf16.html>.